INFORMATION SYSTEMS AND DATABASES–PORTFOLIO ELEMENT 3

CY Batch - 2

S.No	TABLE OF CONTENTS	Pg No.
1	Overview	2
2	Business Rules or Assumption	3
3	Entity Relationship Diagram	5
4	Entity Specification Forms	7
5	SQL Table Creation Scripts	9
6	Sample Data	11
7	Queries	14
8	Stored Procedure	24
9	Security Considerations	26
10	Other Databases	28

OVERVIEW

This project focuses on a scenario for South London University's School of Computing, which regularly hosts guest lectures given by external speakers from different organizations. The institution need a suitable database system that manages every aspect of these events, from requests to execution, in order to speed up and manage this process.

Information regarding staff members, organizations, contacts, speakers, lecture requests, scheduled lectures, and contact history are all supposed to be stored and managed by the system.

In this project, I have:

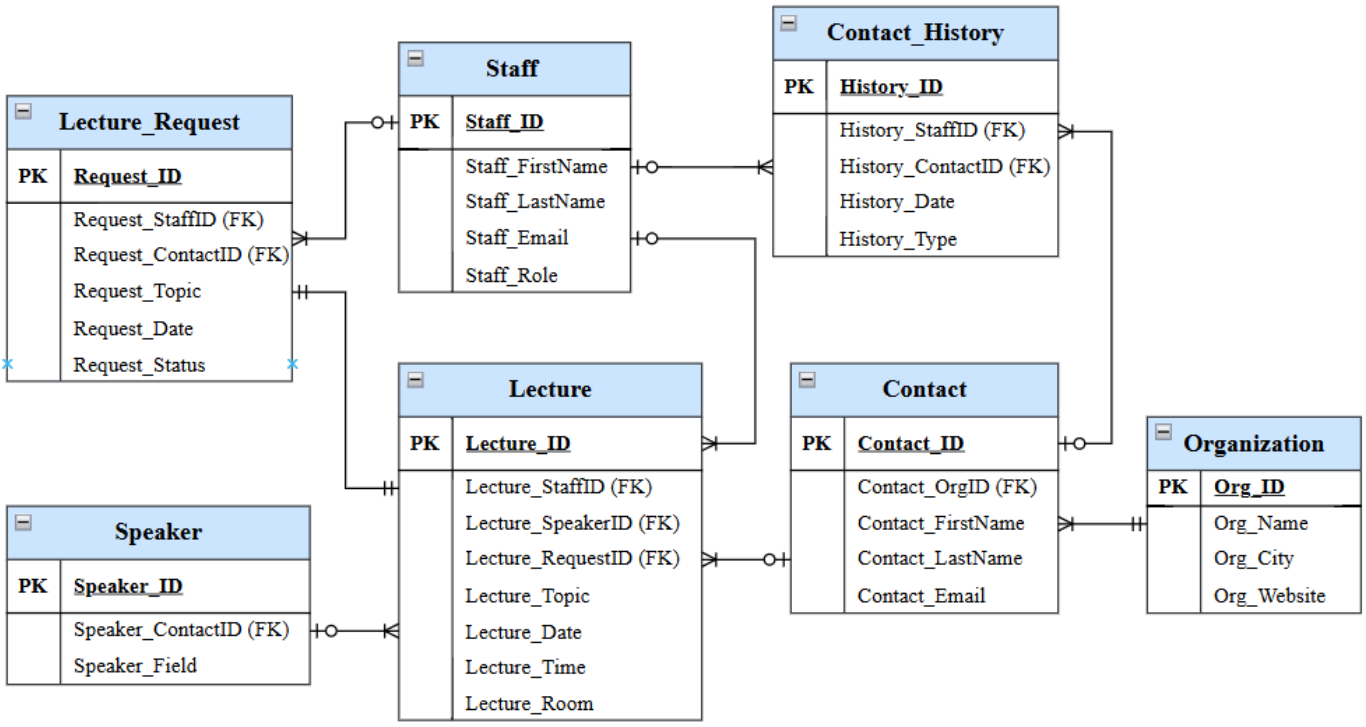
- Evaluated the situation and determined which entities were required.
- Developed an Entity-Relationship Diagram (ERD) that is fully normalized and accurately reflects all of the relationships seen in the real world.
- Based on this ERD, SQL tables were created, making sure they were well-structured and in normalized form.
- To match the real-world entries, data was added to the database.
- SQL queries have been written to support functional requirements, including obtaining speaker information, contact history, or scheduled lectures.
- Role-based access and other security measures were taken into consideration to safeguard confidential employee and contact information.
- Lastly, suitable suggestions for enhancing and expanding the system were addressed.

BUSINESS RULES OR ASSUMPTIONS

No.	Rule / Assumption	Type
1	Every employee in university can draft and submit several requests for lectures from various guest speakers depending on topics and concerns.	Business Rule
2	Every request for a lecture from an employee must be sent to a specific contact from an outside organization rather than to an anonymous or general entity.	Business Rule
3	When a lecture request is accepted, only one lecture event is scheduled; no request can result in more than one lecture.	Business Rule
4	Every lecture must have an assigned member of staff in charge of managing and directing the entire event, from preparation to conclusion.	Business Rule
5	Each lecture will be given by a single guest speaker, guaranteeing topics are assigned clearly and that there are no speaker obligated that overlap.	Business Rule
6	New speakers must first be registered as contacts before they can be allocated as speakers; only contacts who are already saved in the system as contacts may be assigned as speakers.	Business Rule
7	Every contact in the system must be affiliated with a single organization; contacts linked to more than one organization are not permitted.	Business Rule
8	As a reflection of real-world situations where businesses have numerous representatives, an organization might have several contacts listed under it.	Assumption
9	Depending on their area of expertise and requests, a given speaker may give several lectures on various topics or during several semesters.	Assumption
10	Emails, phone conversations, and meetings between university staff and external contacts must all be separately maintained in Contact_History.	Business Rule
11	To ensure authenticity, each lecture request needs to have an update that indicates its present state: pending, approved, or declined.	Business Rule
12	Over time, a contact may be involved in several requests for lectures, indicating several requests to give lectures at different occasions.	Assumption
13	Every Contact_History record is a strictly one-to-one account of communications between a single staff and a single external contact.	Business Rule
14	To maintain data integrity and prevent missing contacts, organizations must be present in the system before contacts referring to them may be added.	Business Rule

15	Unauthorized lectures are not permitted; all scheduled lectures must make reference to a proper lecture request that first created the event's concept.	Business Rule
16	Lecture rooms are distinctly identified in the system by their room numbers, therefore limits duplicate booking and misinterpretation about lecture locations.	Assumption
17	The lecture subject must either meet or relate to with the one specified in Lecture_Request.	Business Rule
18	For the university along with other parties to communicate reliably, all staff and contact email addresses must stick to proper formatting guidelines.	Business Rule
19	Events for lectures must be planned after the date of the request; they cannot take place before to the formal submission of the request.	Business Rule
20	Role-based access control is maintained by restricting the ability to formally accept or reject lecture requests to employees having an administrative role ('admin').	Business Rule

ENTITY RELATIONSHIP DIAGRAM



Relationships

1) **Staff → Lecture_Request**
   - One staff member can have **many** lecture requests.
   - Each lecture request **must** be linked to a staff member.
   - Relationship: One-to-Many (Mandatory)
2) **Staff → Lecture**
   - One staff member can manage **many** lectures.
   - A lecture **can exist** without being assigned to a staff member.
   - Relationship: One-to-Many (Optional)
3) **Staff → Contact_History**
   - One staff member can create **many** contact history records.
   - Each contact history record **must** be linked to a staff member.
   - Relationship: One-to-Many (Mandatory)
4) **Lecture_Request → Lecture**
   - Each lecture is created based on a lecture request.

- Every lecture **must** originate from one lecture request.
- Relationship: One-to-One (Mandatory)

5) **Organization → Contact**
   - One organization can have **many** contacts.
   - Each contact **must** be associated with an organization.
   - Relationship: One-to-Many (Mandatory)

6) **Contact → Contact_History**
   - One contact can appear in **many** contact history records.
   - A contact **may or may not** have any history records.
   - Relationship: One-to-Many (Optional)

7) **Speaker → Lecture**
   - One speaker can deliver **many** lectures.
   - Each lecture **must** be assigned to a speaker.
   - Relationship: One-to-Many (Mandatory)

8) **Contact → Lecture_Request**
   - One contact can submit **many** lecture requests.
   - Each lecture request **must** be submitted by a contact.
   - Relationship: One-to-Many (Mandatory)

ENTITY SPECIFICATION FORMS

ENTITY NAME: **STAFF**
Entity Description: Stores university staff details who manage requests and lectures.

Attribute	Data type	Status	Validation	Example of input
Staff_ID	NUMBER(5)	PK		10001
Staff_FirstName	VARCHAR2(20)	NOT NULL		Aisha
Staff_LastName	VARCHAR2(20)	NOT NULL		Khan
Staff_Email	VARCHAR2(50)	NOT NULL	Valid email format	a.khan@southuni.ac.uk
Staff_Role	VARCHAR2(10)		CHECK ('admin', 'staff')	staff

ENTITY NAME: **ORGANIZATION**
Entity Description: Represents external companies linked to contacts and speakers.

Attribute	Data type	Status	Validation	Example of input
Org_ID	NUMBER(5)	PK		50001
Org_Name	VARCHAR2(50)	NOT NULL		Tech Solutions Ltd
Org_City	VARCHAR2(100)			London
Org_Website	VARCHAR2(100)			www.tech.com

ENTITY NAME: **CONTACT**
Entity Description: People from organizations contacted by staff for speaker arrangements.

Attribute	Data type	Status	Validation	Example of input
Contact_ID	NUMBER(5)	PK		70001
Contact_FirstName	VARCHAR2(20)	NOT NULL		Rohan
Contact_LastName	VARCHAR2(20)	NOT NULL		Patel
Contact_Email	VARCHAR2(50)		Valid email format	r.patel@tech.com
Contact_OrgID	NUMBER(5)	FK	Must exist in Org_ID	50001

ENTITY NAME: **SPEAKER**
Entity Description: Contacts selected to deliver guest lectures on specific topics.

Attribute	Data type	Status	Validation	Example of input
Speaker_ID	NUMBER(5)	PK		80001
Speaker_ContactID	NUMBER(5)	FK	Must exist in Contact_ID	70001
Speaker_Field	VARCHAR2(100)			Cybersecurity

ENTITY NAME: **LECTURE**
Entity Description: Scheduled sessions delivered by speakers to university students.

Attribute	Data type	Status	Validation	Example of input
Lecture_ID	NUMBER(5)	PK		60001
Lecture_Topic	VARCHAR2(100)	NOT NULL		AI in Healthcare
Lecture_Date	DATE	NOT NULL	Future date	20-MAY-2025
Lecture_Time	VARCHAR2(10)			14:00
Lecture_Room	VARCHAR2(10)			B101
Lecture_StaffID	NUMBER(5)	FK	Must exist in Staff_ID	10001
Lecture_SpeakerID	NUMBER(5)	FK	Must exist in Speaker_ID	80001
Lecture_RequestID	NUMBER(5)	FK	Must exist in Request_ID	90001

ENTITY NAME: **LECTURE_REQUEST**
Entity Description: Requests made by staff for guest lectures using specific contacts.

Attribute	Data type	Status	Validation	Example of input
Request_ID	NUMBER(5)	PK		90001
Request_Topic	VARCHAR2(100)	NOT NULL		Blockchain Basics
Request_Date	DATE	NOT NULL		05-APR-2025
Request_Status	VARCHAR2(15)		CHECK ('pending', 'approved', 'declined')	approved
Request_StaffID	NUMBER(5)	FK	Must exist in Staff_ID	10001
Request_ContactID	NUMBER(5)	FK	Must exist in Contact_ID	70001

ENTITY NAME: **CONTACT_HISTORY**
Entity Description: Logs communication between staff and organization contacts over time.

Attribute	Data type	Status	Validation	Example of input
History_ID	NUMBER(5)	PK		30001
History_StaffID	NUMBER(5)	FK	Must exist in Staff_ID	10001
History_ContactID	NUMBER(5)	FK	Must exist in Contact_ID	70001
History_Date	DATE	NOT NULL		12-FEB-2025
History_Type	VARCHAR2(20)		Call, Email, Meeting	Call

SQL TABLE CREATION SCRIPTS

1) STAFF TABLE

```
CREATE TABLE Staff (
    Staff_ID NUMBER(5) PRIMARY KEY,
    Staff_FirstName VARCHAR2(20) NOT NULL,
    Staff_LastName VARCHAR2(20) NOT NULL,
    Staff_Email VARCHAR2(50) NOT NULL,
    Staff_Role VARCHAR2(10),
    CONSTRAINT chk_Staff_Role CHECK (Staff_Role IN ('admin', 'staff')));
```

2) ORGANIZATION TABLE

```
CREATE TABLE Organization (
    Org_ID NUMBER(5) PRIMARY KEY,
    Org_Name VARCHAR2(50) NOT NULL,
    Org_City VARCHAR2(30),
    Org_Website VARCHAR2(100));
```

3) CONTACT TABLE

```
CREATE TABLE Contact (
    Contact_ID NUMBER(5) PRIMARY KEY,
    Contact_FirstName VARCHAR2(20) NOT NULL,
    Contact_LastName VARCHAR2(20) NOT NULL,
    Contact_Email VARCHAR2(50),
    Contact_OrgID NUMBER(5),
    FOREIGN KEY (Contact_OrgID) REFERENCES Organization(Org_ID));
```

4) SPEAKER TABLE

```
CREATE TABLE Speaker (
    Speaker_ID NUMBER(5) PRIMARY KEY,
    Speaker_ContactID NUMBER(5) UNIQUE,
    Speaker_Field VARCHAR2(100),
    FOREIGN KEY (Speaker_ContactID) REFERENCES Contact(Contact_ID));
```

5) LECTURE_REQUEST TABLE

```
CREATE TABLE Lecture_Request (
```

```
    Request_ID NUMBER(5) PRIMARY KEY,
    Request_Topic VARCHAR2(100) NOT NULL,
    Request_Date DATE NOT NULL,
    Request_Status VARCHAR2(15),
    Request_StaffID NUMBER(5),
    Request_ContactID NUMBER(5),
    CONSTRAINT chk_Request_Status CHECK (Request_Status IN ('pending', 'approved',
    'declined')),
    FOREIGN KEY (Request_StaffID) REFERENCES Staff(Staff_ID),
    FOREIGN KEY (Request_ContactID) REFERENCES Contact(Contact_ID));
```

6) LECTURE TABLE

```
CREATE TABLE Lecture (
    Lecture_ID NUMBER(5) PRIMARY KEY,
    Lecture_Topic VARCHAR2(100) NOT NULL,
    Lecture_Date DATE NOT NULL,
    Lecture_Time VARCHAR2(10),
    Lecture_Room VARCHAR2(10),
    Lecture_StaffID NUMBER(5),
    Lecture_SpeakerID NUMBER(5),
    Lecture_RequestID NUMBER(5),
    FOREIGN KEY (Lecture_StaffID) REFERENCES Staff(Staff_ID),
    FOREIGN KEY (Lecture_SpeakerID) REFERENCES Speaker(Speaker_ID),
    FOREIGN KEY (Lecture_RequestID) REFERENCES LectureRequest(Request_ID));
```

7) CONTACT_HISTORY TABLE

```
CREATE TABLE Contact_History (
    History_ID NUMBER(5) PRIMARY KEY,
    History_StaffID NUMBER(5),
    History_ContactID NUMBER(5),
    History_Date DATE NOT NULL,
    History_Type VARCHAR2(20),
    FOREIGN KEY (History_StaffID) REFERENCES Staff(Staff_ID),
    FOREIGN KEY (History_ContactID) REFERENCES Contact(Contact_ID));
```

SAMPLE DATA

1) STAFF

```
INSERT INTO Staff VALUES (10001, 'Aisha', 'Khan', 'a.khan@southuni.ac.uk', 'staff');

INSERT INTO Staff VALUES (10002, 'James', 'Lee', 'j.lee@southuni.ac.uk', 'admin');

INSERT INTO Staff VALUES (10003, 'Sara', 'Ahmed', 's.ahmed@southuni.ac.uk', 'staff');

INSERT INTO Staff VALUES (10004, 'David', 'Nguyen', 'd.nguyen@southuni.ac.uk',
'staff');

INSERT INTO Staff VALUES (10005, 'Emily', 'Wright', 'e.wright@southuni.ac.uk',
'staff');
```

STAFF_ID	STAFF_FIRSTNAME	STAFF_LASTNAME	STAFF_EMAIL	STAFF_ROLE
10001	Aisha	Khan	a.khan@southuni.ac.uk	staff
10002	James	Lee	j.lee@southuni.ac.uk	admin
10004	David	Nguyen	d.nguyen@southuni.ac.uk	staff
10003	Sara	Ahmed	s.ahmed@southuni.ac.uk	staff
10005	Emily	Wright	e.wright@southuni.ac.uk	staff

Fig1.1 – Staff table

2) ORGANIZATION

```
INSERT INTO Organization VALUES (50001, 'Tech Solutions Ltd', 'London',
'www.techsolutions.com');

INSERT INTO Organization VALUES (50002, 'FutureSoft UK', 'Bristol',
'www.futuresoft.co.uk');

INSERT INTO Organization VALUES (50003, 'InnovaTech', 'Manchester',
'www.innovatech.org');
```

ORG_ID	ORG_NAME	ORG_CITY	ORG_WEBSITE
50001	Tech Solutions Ltd	London	www.techsolutions.com
50002	FutureSoft UK	Bristol	www.futuresoft.co.uk
50003	InnovaTech	Manchester	www.innovatech.org

Fig1.2 – Organization table

3) CONTACT

```
INSERT INTO Contact VALUES (70001, 'Rohan', 'Patel', 'r.patel@techsolutions.com',
50001);

INSERT INTO Contact VALUES (70002, 'Sara', 'Ali', 's.ali@futuresoft.co.uk', 50002);

INSERT INTO Contact VALUES (70003, 'Nina', 'White', 'n.white@innovatech.org', 50003);

INSERT INTO Contact VALUES (70004, 'Leo', 'Grant', 'l.grant@futuresoft.co.uk', 50002);

INSERT INTO Contact VALUES (70005, 'Maya', 'Jones', 'm.jones@techsolutions.com',
50001);
```

CONTACT_ID	CONTACT_FIRSTNAME	CONTACT_LASTNAME	CONTACT_EMAIL	CONTACT_ORGID
70001	Rohan	Patel	r.patel@techsolutions.com	50001
70003	Nina	White	n.white@innovatech.org	50003
70002	Sara	Ali	s.ali@futuresoft.co.uk	50002
70004	Leo	Grant	l.grant@futuresoft.co.uk	50002
70005	Maya	Jones	m.jones@techsolutions.com	50001

Fig1.3 – Contact table

4) SPEAKER

```
INSERT INTO Speaker VALUES (80001, 70001, 'Cybersecurity Trends');

INSERT INTO Speaker VALUES (80002, 70002, 'AI and Robotics');

INSERT INTO Speaker VALUES (80003, 70003, 'Data Science in Business');
```

SPEAKER_ID	SPEAKER_CONTACTID	SPEAKER_FIELD
80001	70001	Cybersecurity Trends
80002	70002	AI and Robotics
80003	70003	Data Science in Business

Fig1.4 – Speaker table

5) LECTURE_REQUEST

```
INSERT INTO Lecture_Request VALUES (90001, 'Blockchain Basics', TO_DATE('2025-04-05',
'YYYY-MM-DD'), 'approved', 10001, 70001);

INSERT INTO Lecture_Request VALUES (90002, 'AI for Beginners', TO_DATE('2025-04-10',
'YYYY-MM-DD'), 'pending', 10002, 70002);

INSERT INTO Lecture_Request VALUES (90003, 'Data Ethics', TO_DATE('2025-04-15', 'YYYY-
MM-DD'), 'approved', 10003, 70003);

INSERT INTO Lecture_Request VALUES (90004, 'Cloud Security', TO_DATE('2025-04-20',
'YYYY-MM-DD'), 'approved', 10004, 70005);

INSERT INTO Lecture_Request VALUES (90005, 'IoT Trends', TO_DATE('2025-04-22', 'YYYY-
MM-DD'), 'declined', 10005, 70004);

INSERT INTO Lecture_Request VALUES (91001, 'Test Topic 1', TO_DATE('2025-04-25', 'YYYY-
MM-DD'), 'pending', 10001, 70001);

INSERT INTO Lecture_Request VALUES (91002, 'Test Topic 2', TO_DATE('2025-04-27', 'YYYY-
MM-DD'), 'pending', 10001, 70001);

INSERT INTO Lecture_Request VALUES (91003, 'Test Topic 3', TO_DATE('2025-04-30', 'YYYY-
MM-DD'), 'pending', 10001, 70001);
```

REQUEST_ID	REQUEST_TOPIC	REQUEST_DATE	REQUEST_STATUS	REQUEST_STAFFID	REQUEST_CONTACTID
90003	Data Ethics	4/15/2025	approved	10003	70003
91003	Test Topic 3	5/19/2025	pending	10001	70001
91004	Test Topic 4	5/19/2025	pending	10001	70001
90005	IoT Trends	4/22/2025	declined	10005	70004
91002	Test Topic 2	5/19/2025	pending	10001	70001
90001	Blockchain Basics	4/5/2025	approved	10001	70001
90004	Cloud Security	4/20/2025	approved	10004	70005
90002	AI for Beginners	4/10/2025	approved	10002	70002
91001	Test Topic 1	5/19/2025	pending	10001	70001

Fig1.5 – Lecture_Request table

6) LECTURE

```
INSERT INTO Lecture VALUES (60001, 'Blockchain Basics', TO_DATE('2025-05-01', 'YYYY-MM-DD'), '10:00', 'B101', 10001, 80001, 90001);

INSERT INTO Lecture VALUES (60002, 'AI for Beginners', TO_DATE('2025-05-03', 'YYYY-MM-DD'), '14:00', 'B102', 10002, 80002, 90002);

INSERT INTO Lecture VALUES (60003, 'Data Science in Business', TO_DATE('2025-05-07', 'YYYY-MM-DD'), '12:00', 'B103', 10003, 80003, 90003);
```

LECTURE_ID	LECTURE_TOPIC	LECTURE_DATE	LECTURE_TIME	LECTURE_ROOM	LECTURE_STAFFID	LECTURE_SPEAKERID	LECTURE_REQUESTID
60003	Data Science in Business	5/7/2025	12:00	B103	10003	80003	90003
60002	AI for Beginners	5/3/2025	14:00	B102	10002	80002	90002
60001	Blockchain Basics	5/1/2025	10:00	B101	10001	80001	90001

Fig1.6 – Lecture table

7) CONTACT_HISTORY

```
INSERT INTO Contact_History VALUES (30001, 10001, 70001, TO_DATE('2025-03-01', 'YYYY-MM-DD'),'Call');

INSERT INTO Contact_History VALUES (30002, 10002, 70002, TO_DATE('2025-03-03', 'YYYY-MM-DD'), 'Email');

INSERT INTO Contact_History VALUES (30003, 10003, 70003, TO_DATE('2025-03-05', 'YYYY-MM-DD'), 'Meeting');

INSERT INTO Contact_History VALUES (30004, 10004, 70005, TO_DATE('2025-03-10', 'YYYY-MM-DD'), 'Call');

INSERT INTO Contact_History VALUES (30005, 10005, 70004, TO_DATE('2025-03-15', 'YYYY-MM-DD'), 'Email');
```

HISTORY_ID	HISTORY_STAFFID	HISTORY_CONTACTID	HISTORY_DATE	HISTORY_TYPE
30003	10003	70003	3/5/2025	Meeting
30001	10001	70001	3/1/2025	Call
30100	10001	70001	5/5/2025	System
30002	10002	70002	3/3/2025	Email
30101	10002	70002	5/6/2025	System
30004	10004	70005	3/10/2025	Call
30005	10005	70004	3/15/2025	Email

Fig1.7 – Contact_History table

QUERIES

1) This query helps track contact with the external organization by retrieving each organization's most recent interaction date and the staff person who applied that contact.

- Applying `MAX()` and `GROUP BY`, the query combines tables to determine the most recent conversation dates for an organization and displays which staff person made that contact.
- Helps the institution prioritize follow-ups and prevents them from overlooking important organization by identifying which groups were not contacted in a long time.

```
SELECT C.Contact_FirstName, C.Contact_LastName, O.Org_Name, O.Org_Address
FROM Contact C
JOIN Organization O ON C.Contact_OrgID = O.Org_ID
WHERE O.Org_City IN ('London', 'Bristol');
```

Output-

CONTACT_FIRSTNAME	CONTACT_LASTNAME	ORG_NAME	ORG_CITY
Rohan	Patel	Tech Solutions Ltd	London
Sara	Ali	FutureSoft UK	Bristol
Leo	Grant	FutureSoft UK	Bristol
Maya	Jones	Tech Solutions Ltd	London

Fig 2.1 – output of query 1

2) Report the number of organizations ranked, relative to the highest number of contacts that have participated in guest lecture collaboration with the university.

- This query joins the Organization and Contact tables to count how many contact records have a relationship with each organization at the same time, using the Foreign Key relationship.
- Identifying which organizations contributed the highest amount of external participants, now you have a means to analyze which organizations are the most active in their partnerships, and more importantly which organizations to approach for developing new engagement with and/or appreciation of participation.

```
SELECT O.Org_Name, COUNT(C.Contact_ID) AS Contact_Count
FROM Organization O
JOIN Contact C ON O.Org_ID = C.Contact_OrgID
GROUP BY O.Org_Name
ORDER BY Contact_Count DESC;
```

Output-

ORG_NAME	CONTACT_COUNT
FutureSoft UK	2
Tech Solutions Ltd	2
InnovaTech	1

Fig 2.2 – output of query 2

3) Report the detail of all upcoming lectures, including details about lecture topic, schedule, room, speaker name and name of the member of staff coordinating the session.

- This query joins all required tables (Lecture, Staff, Speaker, and Contact) to capture all speaker and co-ordinator extension with scheduled lecture.
- Overall this provides staff/students some sense of complete schedules for lectures, and allows clarity of description of who is delivering and coordinating each guest lecture in the database.

```
SELECT

    L.Lecture_Topic, L.Lecture_Date, L.Lecture_Time, L.Lecture_Room,

    S.Staff_FirstName || ' ' || S.Staff_LastName AS Staff_Name,

    C.Contact_FirstName || ' ' || C.Contact_LastName AS Speaker_Name

FROM Lecture L

JOIN Staff S ON L.Lecture_StaffID = S.Staff_ID

JOIN Speaker SP ON L.Lecture_SpeakerID = SP.Speaker_ID

JOIN Contact C ON SP.Speaker_ContactID = C.Contact_ID;
```

Output-

LECTURE_TOPIC	LECTURE_DATE	LECTURE_TIME	LECTURE_ROOM	STAFF_NAME	SPEAKER_NAME
Data Science in Business	5/7/2025	12:00	B103	Sara Ahmed	Nina White
AI for Beginners	5/3/2025	14:00	B102	James Lee	Sara Ali
Blockchain Basics	5/1/2025	10:00	B101	Aisha Khan	Rohan Patel

Fig 2.3 – output of query 3

4) Report all lectures coordinated by a member of staff with an 'admin' role and include the lecture topic and full speaker name for clarity.

- This query joins multiple tables (Lecture, Staff, Speaker and Contact) to link lecture scheduling and coordinators and speakers into a single output.
- The WHERE clause filtered only the records with 'admin' role to be assured the output related only to lectures authorized by administrator controlled events.

- It also used string concatenation to deliver full names for both the speaker and coordinator to improve readability, and assist with staff/speaker identity reporting.

```
SELECT
      L.Lecture_ID,
      L.Lecture_Topic,
      S.Staff_FirstName || ' ' || S.Staff_LastName AS Coordinator,
      C.Contact_FirstName || ' ' || C.Contact_LastName AS Speaker
FROM Lecture L
JOIN Staff S ON L.Lecture_StaffID = S.Staff_ID
JOIN Speaker SP ON L.Lecture_SpeakerID = SP.Speaker_ID
JOIN Contact C ON SP.Speaker_ContactID = C.Contact_ID
WHERE S.Staff_Role = 'admin';
```

Output-

LECTURE_ID	LECTURE_TOPIC	COORDINATOR	SPEAKER
60002	AI for Beginners	James Lee	Sara Ali

Fig 2.4 – output of query 4

5) How often was each contact contacted via call, email or face to face, using conditional aggregation for categorized interaction counts.

- This query showed the use of CASE within COUNT() to conditional total lecture requests its a state (pending or approved) per staff.
- The join with the Staff table and bringing in readable staff names instead of IDs was useful, because the report will more meaningful.
- The GROUP BY guarantees counts of requests are getting calculated separately per staff, so we get insight about rates of submission, workloads and capacity, and efficiency of response by administration.

```
SELECT
    c.Contact_FirstName || ' ' || c.Contact_LastName AS Contact,
    COUNT(CASE WHEN ch.History_Type = 'Call' THEN 1 END) AS Call_Count,
    COUNT(CASE WHEN ch.History_Type = 'Email' THEN 1 END) AS Email_Count,
    COUNT(CASE WHEN ch.History_Type = 'Meeting' THEN 1 END) AS Meeting_Count
FROM Contact_History ch
JOIN Contact c ON ch.History_ContactID = c.Contact_ID
GROUP BY c.Contact_FirstName, c.Contact_LastName;
```

Output-

CONTACT	CALL_COUNT	EMAIL_COUNT	MEETING_COUNT
Rohan Patel	1	0	0
Sara Ali	0	1	0
Nina White	0	0	1
Maya Jones	1	0	0
Leo Grant	0	1	0

Fig 2.5 – output of query 5

6) Count each individual staff member's pending and approved lecture requests using conditional aggregation and grouping for accountability.

- Using the CASE statement within COUNT() to count conditional totals of lecture requests by their status (pending or approved) for each staff member.
- The join with the Staff table provides staff names in the output (as opposed to IDs) which makes the report much more readable when reviewing.
- With GROUP BY, the request counts are calculated separately per staff member which explains submission behaviors, workloads and the administrative response time.

```
SELECT
    s.Staff_FirstName || ' ' || s.Staff_LastName AS staff_name,
    COUNT(CASE WHEN lr.Request_Status = 'pending' THEN 1 END) AS pending_requests,
    COUNT(CASE WHEN lr.Request_Status = 'approved' THEN 1 END) AS approved_requests
FROM Lecture_Request lr
JOIN Staff s ON lr.Request_StaffID = s.Staff_ID
GROUP BY s.Staff_FirstName, s.Staff_LastName;
```

Output-

STAFF_NAME	PENDING_REQUESTS	APPROVED_REQUESTS
James Lee	0	1
Aisha Khan	0	1
David Nguyen	0	1
Emily Wright	0	0
Sara Ahmed	0	1

Fig 2.6 – output of query 6

7) Return all of the lecture requests that are not explicitly rejected, included the contact person, (their organization), and the staff member who submitted the request.

- This query focused on all requests that were still active (i.e., not rejected but pending or approved) and the filter Request_Status != 'rejected'.
- It joins four tables (Lecture_Request, Contact, Organization, and Staff), which will help us better understand where the request came from, and where it currently is at.
- By sorting first by request status, then by date (159), administrators can see what requests are still open, or are newly updated.

```
SELECT
    LR.Request_ID,
    LR.Request_Topic,
    LR.Request_Date,
    LR.Request_Status,
    C.Contact_FirstName || ' ' || C.Contact_LastName AS Contact_Name,
    C.Contact_Email,
    O.Org_Name AS Organization,
    S.Staff_FirstName || ' ' || S.Staff_LastName AS Requested_By
FROM Lecture_Request LR
JOIN Contact C ON LR.Request_ContactID = C.Contact_ID
JOIN Organization O ON C.Contact_OrgID = O.Org_ID
JOIN Staff S ON LR.Request_StaffID = S.Staff_ID
WHERE LR.Request_Status != 'rejected'
ORDER BY LR.Request_Status, LR.Request_Date DESC;
```

Output-

REQUEST_ID	REQUEST_TOPIC	REQUEST_DATE	REQUEST_STATUS	CONTACT_NAME	CONTACT_EMAIL	ORGANIZATION	REQUESTED_BY
90004	Cloud Security	4/20/2025	approved	Maya Jones	m.jones@techsolutions.com	Tech Solutions Ltd	David Nguyen
90003	Data Ethics	4/15/2025	approved	Nina White	n.white@innovatech.org	InnovaTech	Sara Ahmed
90002	AI for Beginners	4/10/2025	approved	Sara Ali	s.ali@futuresoft.co.uk	FutureSoft UK	James Lee
90001	Blockchain Basics	4/5/2025	approved	Rohan Patel	r.patel@techsolutions.com	Tech Solutions Ltd	Aisha Khan
90005	IoT Trends	4/22/2025	declined	Leo Grant	l.grant@futuresoft.co.uk	FutureSoft UK	Emily Wright

Fig 2.7 – output of query 7

8) Determine what the most contacted person by each staff member was by looking at the amount of recorded contacts that was in the contact history.

- This query used GROUP BY History_StaffID and History_ContactID to develop a summarization of all contacts that a staff member had contact with their contacts.

- The HAVING clause guaranteed we were excluding non-max records because only the most contacted position of the contact history for each staff is relevant.
- This allows for meaningful analysis of communications patterns to help differentiate the key outside collaborates engaged. It also allows one to understand the extent of collaboration during the guest lecture engagement process.

```
SELECT History_StaffID, History_ContactID, COUNT(*) AS Contact_Count

FROM Contact_History

GROUP BY History_StaffID, History_ContactID

HAVING COUNT(*) = (

    SELECT MAX(Cnt) FROM (

        SELECT COUNT(*) AS Cnt

        FROM Contact_History

        GROUP BY History_StaffID, History_ContactID ) );
```

Output-

HISTORY_STAFFID	HISTORY_CONTACTID	CONTACT_COUNT
10002	70002	2
10001	70001	2

Fig 2.8 – output of query 8

9) Report all contacts whose organizations are either 'London' or 'Bristol' using the IN operator to match against multiple values.

- This query brings back contact names along with the organization details by using a join to the Contact and Organization tables using the Foreign Key relationship.
- The result is filtered against what city the organization is located in, using the IN operator which means instead of checking against multiple values like London and Bristol, it simply goes in and has the option to go out against as many values as you care to include.

```
SELECT C.Contact_FirstName, C.Contact_LastName, O.Org_Name, O.Org_Address

FROM Contact C

JOIN Organization O ON C.Contact_OrgID = O.Org_ID

WHERE O.Org_City IN ('London', 'Bristol');
```

Output-

CONTACT_FIRSTNAME	CONTACT_LASTNAME	ORG_NAME	ORG_CITY
Rohan	Patel	Tech Solutions Ltd	London
Sara	Ali	FutureSoft UK	Bristol
Leo	Grant	FutureSoft UK	Bristol
Maya	Jones	Tech Solutions Ltd	London

Fig 2.9 – output of query 9

10) Categorise All Lecture Requests by Urgency, based on the request dates, using CASE, and showing which staff member submitted it.

- This query uses CASE to dynamically label every lecture request based on how far away its date is from today (SYSDATE).
- This query is joining the Lecture_Request with the Staff to show the requester as a full name in the report, so that a person may assign verbs or follow-ups and what to prioritise.
- This helps schedule staff to look at which of their lecture requests are urgent, upcoming, missed or safe for the future. This will ultimately allow for the most effective strategic planning.

```
SELECT

    lr.Request_ID,

    lr.Request_Topic,

    s.Staff_FirstName || ' ' || s.Staff_LastName AS Staff_Member,

    CASE

        WHEN lr.Request_Date < SYSDATE THEN 'Missed Deadline'

        WHEN lr.Request_Date = SYSDATE THEN 'Due Now'

        WHEN lr.Request_Date - SYSDATE <= 7 THEN 'Approaching Deadline'

        WHEN lr.Request_Date - SYSDATE <= 30 THEN 'Upcoming Deadline'

        ELSE 'Distant Deadline'

    END AS Deadline_Category

FROM Lecture_Request lr

JOIN Staff s ON lr.Request_StaffID = s.Staff_ID;
```

Output-

REQUEST_ID	REQUEST_TOPIC	STAFF_MEMBER	DEADLINE_CATEGORY
90001	Blockchain Basics	Aisha Khan	Missed Deadline
90002	AI for Beginners	James Lee	Missed Deadline
90004	Cloud Security	David Nguyen	Missed Deadline
90003	Data Ethics	Sara Ahmed	Missed Deadline
90005	IoT Trends	Emily Wright	Missed Deadline

Fig 2.10 – output of query 10

11) Create a view called approved_lectures_view that shows only the approved lectures along with details of speakers and topics. These retrieves will join together various tables of lecture related data so see the full picture of "lecture".

- This query defines a view joining data from the Lecture, Lecture_Request, Speaker, and Contact tables in order to build a simpler, reusable dataset.
- This query specifies that only records with Request_Status = 'approved' will be retained in the view. The approved corresponds to the approval and confirmation of lecturers through to a request process.
- This simplifies future reporting and front-end access of the sourced data by removing repetitive queries each time a complex join is required and storing it in a simple view for access as a virtual table.

```
CREATE VIEW approved_lectures_view AS
SELECT
    l.Lecture_ID,
    l.Lecture_Topic,
    l.Lecture_Date,
    s.Speaker_Field,
    c.Contact_FirstName || ' ' || c.Contact_LastName AS Speaker_Name
FROM Lecture l
JOIN Lecture_Request lr ON l.Lecture_RequestID = lr.Request_ID
JOIN Speaker s ON l.Lecture_SpeakerID = s.Speaker_ID
JOIN Contact c ON s.Speaker_ContactID = c.Contact_ID
WHERE lr.Request_Status = 'approved';
SELECT * FROM approved_lectures_view;
```

Output-

LECTURE_ID	LECTURE_TOPIC	LECTURE_DATE	SPEAKER_FIELD	SPEAKER_NAME
60003	Data Science in Business	5/7/2025	Data Science in Business	Nina White
60002	AI for Beginners	5/3/2025	AI and Robotics	Sara Ali
60001	Blockchain Basics	5/1/2025	Cybersecurity Trends	Rohan Patel

Fig 2.11 – output of query 11

12) Change the status of a lecture request from "pending" to "approved" and log the action in a contact history record, then commit them together.

- This transaction updates a status of a lecture request and logs the action for audit purposes in Contact_History.
- The COMMIT is a logical action since we want to save both data changes only if both are successful, which preserves data integrity
- This replicates a real world transactional process, example, updating a status and logging the communication.

```
BEGIN

    -- Step 1: Approve a lecture request

    UPDATE Lecture_Request

    SET Request_Status = 'approved'

    WHERE Request_ID = 90002;

    -- Step 2: Log this approval in Contact_History

    INSERT INTO Contact_History (

        History_ID,

        History_StaffID,

        History_ContactID,

        History_Date,

        History_Type

    ) VALUES (

        30101, 10002, 70002, TO_DATE('2025-05-06', 'YYYY-

        MM-DD'),'System');

    -- Step 3: Commit the transaction

    COMMIT;

END;

SELECT*FROM Lecture_Request

SELECT*FROM Contact_History
```

Output-

REQUEST_ID	REQUEST_TOPIC	REQUEST_DATE	REQUEST_STATUS	REQUEST_STAFFID	REQUEST_CONTACTID
90003	Data Ethics	4/15/2025	approved	10003	70003
90005	IoT Trends	4/22/2025	declined	10005	70004
90001	Blockchain Basics	4/5/2025	approved	10001	70001
90004	Cloud Security	4/20/2025	approved	10004	70005
90002	AI for Beginners	4/10/2025	approved	10002	70002

HISTORY_ID	HISTORY_STAFFID	HISTORY_CONTACTID	HISTORY_DATE	HISTORY_TYPE
30003	10003	70003	3/5/2025	Meeting
30001	10001	70001	3/1/2025	Call
30100	10001	70001	5/5/2025	System
30002	10002	70002	3/3/2025	Email
30101	10002	70002	5/6/2025	System
30004	10004	70005	3/10/2025	Call
30005	10005	70004	3/15/2025	Email

Fig 2.12 – output of query 12

STORED PROCEDURE – TRIGGER

Use Trigger to monitor pending lecture requests for per contact by resting if a new insert into the Lecture_Request table moves the count from 3 to 4

- This trigger enforces that no contact would have numerous lectures requests pending. The trigger will fire after every insert on the Lecture_Request table and check to see how many requests are pending for that same contact.
- If the pending requests are greater than 3, a custom exception will raise and a message will appear in the DBMS_OUTPUT.
- This will help maintain a distribution of lecture requests and cause a warning about too many pending items and any processing will take too long!

```
CREATE OR REPLACE TRIGGER tr_check_pending_requests
AFTER INSERT ON Lecture_Request
FOR EACH ROW
DECLARE
  v_pending_count  NUMBER;
  ex_too_many_pending  EXCEPTION;
BEGIN
  SELECT COUNT(*)
    INTO v_pending_count
    FROM Lecture_Request
    WHERE Request_ContactID = :NEW.Request_ContactID
      AND Request_Status = 'pending';

  IF v_pending_count > 3 THEN
    RAISE ex_too_many_pending;
  END IF;

EXCEPTION
  WHEN ex_too_many_pending THEN
    dbms_output.put_line('Contact ID ' || :NEW.Request_ContactID || ' has too many
pending requests.');
END;
```

Output-

Fig 3.1 – output of query

Trigger Validation

Input –

```
INSERT INTO Lecture_Request VALUES (
  91004, 'Test Topic 4', SYSDATE, 'pending', 10001, 70001
);
```

Ouput -

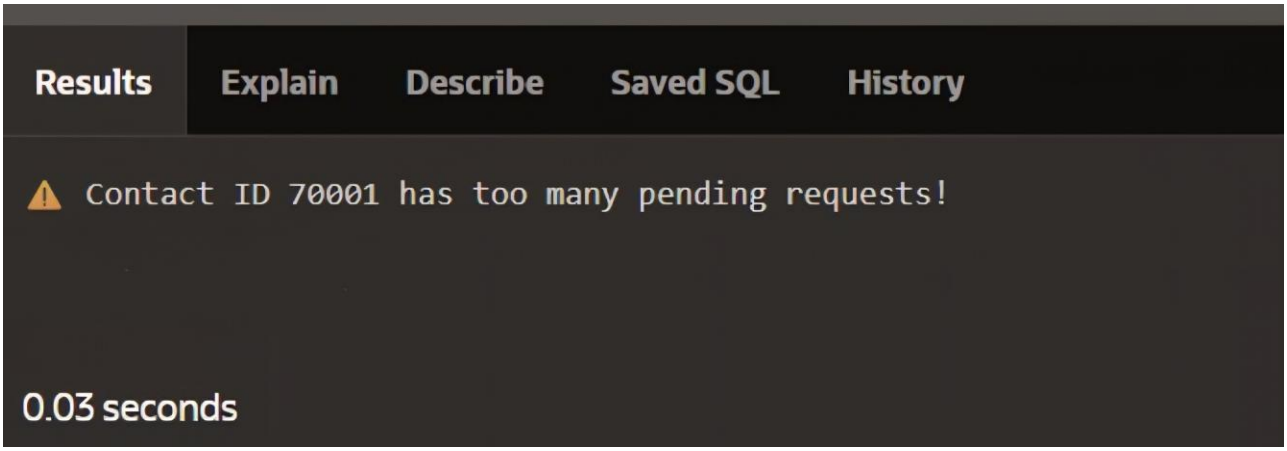


Fig 3.2 – output of query

SECURITY CONSIDERATIONS

Addressing security in a university database is a vital part of the protection of sensitive information such as staff and other external contact details, as well as lecture related information. We often store a considerable amount of confidential business records, and without suitable security checks these records could be exposed to external risk by someone's unauthorized access, breaches of divorce or by the unintentional misuse of that information internally. The installation of reasonable security controls would ideally preserve the confidentiality, integrity, and availability of that data while also preserving trust with the persons, organizations and entities that are impacted and will limit risks of any violations against data protection regulation. Security is not only a technical issue, but even more so a procedural operation that acts as a foundation for operating securely across a variety of university academic and administrative settings. Security is not a one-time charge, but is instead an ongoing process that must be continually updated and overseen.

A list of database countermeasures is presented below to install a strong and dependable security model for the South London University's information management system.

1. Authorization and Access Control

Users must only be authorized to access or alter data by established assigned roles. Role Based Access Control (RBAC) will allow personnel, admin, or other users to classify the specific objects of that access for use. For instance, the class assigned can allow only the general staff to view lecture requests while the administrator can allow others to approve or decline those requests.

a) Use of Roles

Example - Create a role for staff members so that their contact and lecture requests information is minimal.

```
CREATE ROLE staff_role;
```

Example - Provide appropriate authorizations for that role:

```
GRANT SELECT, UPDATE (Request_Status) ON Lecture_Request TO staff_role;
```

```
GRANT SELECT ON Contact TO staff_role;
```

This means that the staff has the ability only to view and update request status, but cannot insert or delete rows.

b) Granting System Privileges

Example - You can grant system privileges to users using the GRANT command:

This creates secure basic access to the user while still applying their role privileges to your system.

2. Auditing, and Other Procedures Related to it

Auditing is a security mechanism of critical importance in a database that records the activity an affected record/ data source undergoes in your database. This tool creates accountability, for your audit trail will reveal who accessed and/or altered the data, when and what was done to the data.

In the south London university, auditing can support information transparency and help establish data governance processes that support the investigation of unauthorized or unacceptable behaviour and other violations of data protection and related policies. It can help administrators trace problems, provide user compliance oversight, and generate reporting to support any security audit. The audit trail creates an additional record of a user's interaction with the database and continues to build institutional trust and support secure academic information governance by keeping a record of a specific interaction; these requirements are crucial in a multi-user academic context.

3. Data encryption

Data encryption is necessary for South London University to protect highly-sensitive information (student data, employee passwords, and personal contact details) from threats and unauthorised access. Data encryption converts normal readable data into encryption format – which will not be readable to an unauthorised threat actor even if the database is compromised.Although encryption is a data security requirement for confidentiality when data is being transferred and stored, this also includes the encryption of personal email addresses and email requests. Data encryption protects privacy requirements and meets data protection legislation requirements such as that set out by GDPR.

Specific formats can be encrypted at the column level (this, for example, can help encrypt passwords and emails) and can also encrypt data when backing-up information (this can help fortify and strengthen the security of the database in various exposure scenarios - itself an additional value add).

4. Backing-up data

For successfully protecting against data losses of any kind – data back-ups are an essential requirement for the protection of South London University academic requests whereby restoration or recovery is possible in the event of a system failure, data theft, or data corruption.Backing-up the database enables South London University to preserve, in its entirety, an academic record of staff assignments made to requests; contact history made during requests; critical data points such as meeting requests and emails scheduled with staff. Back-ups can be successfully scheduled daily or weekly, and can be securely stored off community or cloud storage.

In the event that a portfolio is corrupt, lost or was deleted or compromised in some form such as through a ransomware cyber-attack, a data back-up provides the opportunity to restore status-quo operations without any data being compromised. Back-Up options should implement an automated option by utilising backup procedures and testing the recovery procedures regularly, thus protecting the integrity of the academic integrity and records of data that guide the academic and administrative affairs of the university.

OTHER DATABASES

Database technology provides the fundamental basis for data management and retention in organizations permitting large amounts of data to be retained securely with multi-user support, complex querying and reporting, with safety and data integrity. Educational units, for example, South London University, uses a database for the retention of organized academic records/staff and student records, planning lectures, and recording interactions and communications at the university.

South London University uses Oracle Database - the relational model of database technology - as part of its degree program. The basic structure of a relational database is tables, containing rows and columns. Each table in this simplified example appears to relate to some real world entity such as Staff, Lecture_Request, Contact and Speaker. Typically, each row represents a unique record in a table. Relationships in between tables are created through primary keys (which identify records as unique) and foreign keys (which point to the primary keys of one table in another table). Relationships guarantee logical and consistent access between data in one table to another, and assist in the related area of integrity of data.

The relational model supports integrated querying **Structured Query Language (SQL).** SQL is a powerful querying language; it retrieves, updates, and manipulates structured data effectively. The relational model also supports a concept called normalization which helps in eliminating a lot of redundancy and organizes data using multiple related tables. This maintains the integrity of the information, reduces duplication and efficient management of the database. For example, if all contact details were retained in a single separate Contact table, and the connection to this table was made with a foreign key to the Lecture_Request table, and vice versa. Then any updates and queries could be carried out much more easily and more effectively with less errors.

Limitations of Relational Database Models –

- Problems with scalability: RDBMS may have trouble with large horizontal scaling among separated systems.
- Rigid schema: Modifications to table structure may be challenging or disturbing
- Unstructured data handling: Not suitable for text, photos, videos, etc.
- Blocks performance - Large joins or systems with a lot of traffic can cause performance problems.

While the university is utilizing the relational model provided by Oracle, it is worth highlighting that Oracle Database, like any other modern database that is well designed, is capable of supporting the **object-relational model**. For example, a university could define a Lecture_Material type that has properties such as file name, format, size, and description; or create multimedia presentations and videos by embedding various media formats into a video.

- Object-relational databases are suited to managing complex or multimedia data, both of which are increasingly seen and experienced in an **academic/institutional environment**.
- In that sense, the object-relational model enables a more natural approach to data modelling to support more modern learning systems, digital libraries, and integrated learning environments, representing the nested structure of an individual (or entity) as they exist in the real world within the database.

In conclusion, and based on the defined and hierarchical data structures South London University wishes to pursue with Oracle's relational model, they are correct. Nonetheless, they should also consider the limited use of object-relational capabilities for greater versatility and flexibility, particularly with multimedia objects and more advanced or inherent forms of data processing which will better position their Oracle Database to successfully manage the increasingly multifaceted and diverse nature of academia and the environments with its ever-changing demands.